

X.509 MQTT Auto-Provisioning with mTLS - Analysis Report

This report analyzes the implementation requirements for **X.509 MQTT Auto-Provisioning** on ESP32 devices within the current Coolify-managed OpenRemote deployment.

1. Analysis of Current MQTT Interface (Port 8883)

Current State Assessment

Based on `docker-compose.coolify.yml`:

- **Port Exposure:** Port 8883 is directly exposed from the `proxy` container via `ports: - "8883:8883"`.
- **Routing Mechanism:** It bypasses Traefik (TCP Passthrough). This is the correct foundation for mTLS because Traefik is only handling HTTP/HTTPS labels for port 80/443.
- **HAProxy Configuration:** Port 8883 is bound to `frontend mqtt`, which currently uses `mode tcp` and forwards to `backend mqtt_backend` (OpenRemote Manager).

[!CAUTION]

Protocol Mismatch Identified: The current setup forwards encrypted port 8883 traffic to the Manager's plain-text port 1883. Without HAProxy terminating TLS or the Manager listening for SSL on that port, connections will fail.

2. HAProxy Configuration for mTLS

To support Mutual TLS (mTLS), the internal HAProxy must be configured to request and verify client certificates.

Required Modification

The `frontend mqtt` section in the HAProxy configuration needs to be updated as follows:

```
frontend mqtt
  bind *:8883 ssl crt /certs/mqtt-server-combined.pem ca-file
/certs/ca.pem verify required
  mode tcp
  default_backend mqtt_backend
```

Key Parameters:

- `ssl`: Enables TLS termination at HAProxy.
- `crt ...`: Provides the server certificate to the ESP32 (Server Auth).

- **ca-file** ...: Provides the Root CA used to validate the ESP32's certificate (Client Auth).
 - **verify required**: Forces the client to provide a valid certificate signed by the specified CA.
-

3. Certificate Strategy Recommendation

Found Infrastructure

The project already contains a private PKI in `secrets/certs/`:

- **Root CA**: `ca.pem` (Self-signed, 4096-bit RSA).
- **Device Certs**: `device01.pem`, etc.

Strategic Decision: Strategy B (Dedicated MQTT CA)

Recommendation: Use the existing dedicated Root CA for MQTT instead of Let's Encrypt.

Rationale:

1. **mTLS Control:** Let's Encrypt cannot issue the specific client certificates needed for your ESP32 devices.
 2. **Persistence:** Self-signed CAs with long validity (e.g., your current 2-year CA) are more stable for hardcoded IoT firmware than 90-day public certs.
 3. **Efficiency:** Mounting the existing `ca.pem` is highly compatible with the current `docker-compose.yml` volume structure.
-

4. Implementation Steps

Step 1: Prepare the Server Certificate

HAProxy requires a "combined" file containing both the certificate and the private key.

```
# Combine the server cert and key (Run this in secrets/certs/)
cat mqtt-server.pem mqtt-server.key > mqtt-server-combined.pem
```

Step 2: Mount Certificates in `docker-compose.coolify.yml`

Update the `proxy` service to include the certificate directory:

```
services:
  proxy:
    # ... other config ...
    volumes:
      - proxy-data:/deployment
      - ./secrets/certs:/certs:ro # Mount the certs directory
```

Step 3: Configure OpenRemote Manager

1. Access the **Manager UI** -> **Provisioning**.
2. Create an **X.509 MQTT Provisioning** record.
3. Upload **ca.pem** as the trusted CA.
4. Set the **Asset Template** so the Manager knows how to create the device once it connects.

5. Summary Table

Component	Status	Required Action
Port 8883	✓ Exposed	None (TCP Passthrough confirmed)
Server Auth	× Insecure	Generate & Bind server cert in HAProxy
Client Auth	× Disabled	Set verify required in HAProxy
PKI Files	✓ Found	Mount ./secrets/certs to /certs
Auto-Provisioning	⚠ Pending	Enable X.509 flow in Manager UI